
IMPLEMENTACIÓN DE SISTEMA DE GESTIÓN DE COLAS PARA ATENCIÓN AL CLIENTE CON ESTRUCTURAS DE DATOS DINÁMICAS

202100305 – José Alexander López López

Resumen

Este ensayo presenta el diseño e implementación de un sistema de gestión de colas para atención al cliente desarrollado con Python, empleando estructuras de datos dinámicas propias como alternativa a las estructuras nativas del lenguaje. Se analiza cómo la utilización de Tipos de Datos Abstractos (TDA) como listas enlazadas, colas, pilas y tablas hash implementadas desde cero permite una gestión eficiente del flujo de clientes y recursos en entornos empresariales.

Se examina la arquitectura de software orientada a objetos aplicada, las ventajas del manejo de archivos XML para configuración del sistema y el uso de Graphviz para visualización de estructuras. Además, se destacan las implicaciones técnicas y económicas para empresas que requieren optimizar procesos de atención presencial.

Palabras clave

Estructuras dinámicas, Python, TDA, atención al cliente, colas

Abstract

This essay presents the design and implementation of a queue management system for customer service developed with Python, using custom dynamic data structures as an alternative to the language's native structures. It analyzes how the use of Abstract Data Types (ADT) such as linked lists, queues, stacks, and hash tables implemented from scratch allows for efficient management of customer flow and resources in business environments.

The applied object-oriented software architecture, the advantages of using XML files for system configuration, and the use of Graphviz for structure visualization are examined. Furthermore, technical and economic implications for companies that need to optimize face-to-face service processes are highlighted.

Keywords

Dynamic structures, Python, ADT, customer service, queues

Introducción

La gestión eficiente de la atención al cliente representa una ventaja competitiva significativa para empresas que ofrecen servicios presenciales. Largas esperas, desorganización en la asignación de recursos o información imprecisa sobre tiempos de atención generan experiencias negativas y pérdidas económicas considerables. El presente trabajo aborda esta problemática mediante el desarrollo de un sistema prototipo que permite administrar puntos de atención, escritorios de servicio y flujo de clientes.

Este proyecto se desarrolla utilizando el paradigma de programación orientada a objetos en Python, implementando estructuras de datos dinámicas propias y aplicando principios de abstracción para modelar la solución. Se prescinde intencionalmente del uso de estructuras nativas del lenguaje como listas, diccionarios o conjuntos para profundizar en la comprensión fundamental de los algoritmos y estructuras que sustentan aplicaciones de software complejas.

El objetivo principal es demostrar cómo el diseño adecuado de estructuras de datos y algoritmos impacta directamente en el rendimiento, escalabilidad y mantenibilidad de sistemas de gestión empresarial.

Desarrollo del tema

a. Modelado del problema mediante TDA y estructuras dinámicas

El modelado del sistema requiere representar entidades como empresas, puntos de atención, escritorios de servicio, transacciones y clientes. Para

ello, se implementó un conjunto de Tipos de Datos Abstractos (TDA) que permiten gestionar estas entidades de manera eficiente sin recurrir a las estructuras nativas de Python.

La implementación incluye estructuras fundamentales como el Nodo, componente básico para construir estructuras más complejas como ListaEnlazada, Cola, Pila y TablaHash. Este enfoque permite una mayor comprensión de los mecanismos internos y ofrece control preciso sobre la gestión de memoria, aspecto crítico en sistemas que manejan flujos variables de usuarios.

```
# Nodo básico para listas enlazadas
class Nodo:
    def __init__(self, dato=None):
        self.dato = dato
        self.siguiente = None
        self.anterior = None # Para listas doblemente enlazadas

# Lista enlazada simple
class ListaEnlazada:
    def __init__(self):
        self.cabeza = None
        self cola = None
        self.tamaño = 0
```

Figura 1: Modelado de problema mediante TDA
Fuente: Elaboración propia, 2025

La lista enlazada constituye la estructura base para muchas operaciones del sistema. Su implementación proporciona operaciones de inserción, eliminación y búsqueda con complejidad $O(n)$ en el peor caso, pero también permite inserción al final en $O(1)$ gracias al mantenimiento de la referencia a la cola. Esta eficiencia es crucial para el manejo de colecciones dinámicas de entidades empresariales.

La Cola, implementada sobre la lista enlazada, gestiona el principio FIFO (First In, First Out) requerido para la atención secuencial de clientes, garantizando justicia en el orden de atención. La Pila, con su comportamiento LIFO (Last In, First Out), se utiliza para la gestión de escritorios de servicio, permitiendo activar y desactivar escritorios según demanda.

La implementación de TablaHash proporciona búsquedas en tiempo casi constante $O(1)$, crucial para recuperar rápidamente información de empresas, transacciones o clientes por identificadores únicos, mejorando notablemente el rendimiento del sistema en comparación con búsquedas secuenciales.

b. Arquitectura del sistema y patrones de diseño

El sistema se organiza siguiendo una arquitectura modular que separa claramente las responsabilidades entre el modelo de datos, la lógica de negocio y la interfaz de usuario. Esta separación facilita el mantenimiento y la extensión del sistema, permitiendo modificar componentes específicos sin afectar al resto.

El modelo de datos está representado por clases como Empresa, PuntoAtencion, EscritorioServicio, Transaccion y Cliente, que encapsulan tanto datos como comportamiento relacionado con cada entidad. La lógica de negocio se concentra en la clase SistemaAtencion, que coordina las operaciones entre entidades y gestiona el estado global del sistema.

```
class PuntoAtencion:
    def __init__(self, id_punto, nombre, direccion):
        self.id = id_punto
        self.nombre = nombre
        self.direccion = direccion
        self.escriptorios = ListaEnlazada()
        self.cola_clientes = Cola()
        self.pila_escriptorios_activos = Pila()
```

Figura 2: Arquitectura del sistema y patrones de diseño
Fuente: Elaboración propia, 2025

Se implementa el patrón de diseño Facade a través de la clase SistemaAtencion, que simplifica la interacción con el sistema complejo proporcionando una interfaz unificada. Este patrón es particularmente útil para la integración con la interfaz gráfica, reduciendo el acoplamiento entre componentes.

La gestión de los escritorios de servicio utiliza una variante del patrón State, donde cada escritorio

puede estar en estados como INACTIVO, ACTIVO u OCUPADO, con comportamientos diferentes según su estado actual. Esta implementación facilita la gestión del ciclo de vida de los escritorios y su interacción con los clientes.

c. Procesamiento de archivos XML y generación de reportes

El sistema utiliza archivos XML para la configuración inicial y la carga de datos de prueba. Esta elección ofrece ventajas como la estructuración jerárquica natural para representar entidades relacionadas y la facilidad de modificación manual o programática.

```
def cargar_configuracion(self, ruta_archivo):
    try:
        tree = ET.parse(ruta_archivo)
        root = tree.getroot()

        if root.tag != 'listaEmpresas':
            raise ValueError("Formato de archivo XML incorrecto.")

        for empresa_elem in root.findall('empresa'):
            # Procesamiento del XML...
```

Figura 3: Procesamiento de archivos XML y reportes
Fuente: Elaboración propia, 2025

El procesamiento XML se realiza mediante la biblioteca estándar ElementTree, que proporciona un API simple y eficiente para analizar y generar documentos XML. La clase ProcesadorXML encapsula esta funcionalidad, separando las preocupaciones de procesamiento de datos de entrada del resto del sistema.

Para la visualización de estructuras como la cola de espera y los escritorios de servicio, se utiliza Graphviz, una herramienta que permite generar representaciones gráficas de estructuras de datos. Esta integración facilita la comprensión del estado del sistema tanto para operadores como para administradores.

```
def generar_dotCola(self):  
    if not self.punto_actual:  
        return ""  
  
    dot = graphviz.Digraph(comment='Cola de Espera')  
    # Generación del diagrama...  
    return dot.source
```

Figura 4: Modelado de problema mediante TDA
Fuente: Elaboración propia, 2025

Los reportes generados muestran estadísticas clave como tiempos promedio de espera, tiempos de atención y distribución de clientes, proporcionando información valiosa para la toma de decisiones operativas y estratégicas.

d. Interfaz gráfica y experiencia de usuario

La interfaz gráfica se desarrolló utilizando Tkinter, biblioteca estándar de Python para interfaces gráficas. La interfaz se organiza en pestañas que separan las funcionalidades de configuración del sistema y la operación de puntos de atención, proporcionando una experiencia de usuario intuitiva y funcional.

La clase VentanaPrincipal actúa como controlador principal, gestionando la interacción entre la interfaz gráfica y el sistema subyacente. Se implementan diálogos modales para operaciones como la creación de empresas, puntos de atención o la solicitud de atención para clientes.

La visualización en tiempo real del estado del sistema, incluyendo escritorios activos, clientes en espera y estadísticas de operación, permite a los operadores tomar decisiones informadas para optimizar la atención al cliente. La capacidad de simular la actividad completa del punto de atención facilita además la planificación y evaluación de escenarios operativos.

Un aspecto destacable es la retroalimentación visual mediante diagramas generados con Graphviz, que complementa las estadísticas numéricas con representaciones gráficas de fácil interpretación,

mejorando la comprensión del estado del sistema para usuarios con diferentes perfiles técnicos.

Conclusiones

La implementación de estructuras de datos dinámicas propias proporciona un control preciso sobre el comportamiento del sistema, ofreciendo ventajas significativas frente a las estructuras nativas de Python para soluciones empresariales específicas. El enfoque basado en TDA y programación orientada a objetos resultó en una arquitectura modular y mantenible que facilita adaptaciones futuras.

La integración de archivos XML para configuración y Graphviz para visualización mejoró considerablemente la usabilidad del sistema, permitiendo a usuarios técnicos y no técnicos entender el estado operativo de forma intuitiva.

Se evidencia que la selección adecuada de estructuras de datos impacta directamente en el rendimiento y escalabilidad de sistemas de gestión, generando beneficios técnicos y económicos para organizaciones que optimizan sus procesos de atención al cliente.

Referencias bibliográficas

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. Wiley.